

Thesis B — Progress Update

Comparison and Implementation of Data Compression Methods
for SoC-based Radar Systems

Jordan Mohindra (z5418455) · Supervisor: A/Prof. Beena Ahmed · 17 September 2026

The short version

Since the Thesis A report I have built, verified and characterised complete hardware implementations of **two** lossless radar compression algorithms — DRHE and LPC — on the Kintex Ultra-Scale+ FPGA, each with a matching decompressor. Both reconstruct real radar data perfectly: zero error across 157 million samples, confirmed three separate ways.

Then I audited the results, and found something I was not expecting.

The main finding. Applied the way the literature describes them, *neither predictor was doing anything useful*. Running the same data through the same entropy coder with *no prediction at all* compressed **better** than either algorithm — 3.489 against 3.319 (DRHE) and 3.397 (LPC). Both predictors were making the data measurably harder to compress.

The cause is that our radar is a 12-transmitter TDM-MIMO cascade, and the transmitters are interleaved along the axis the algorithms predict along. “The previous ramp” is a different physical antenna eleven times out of twelve. Predicting from the previous ramp of the *same* transmitter fixes it, and lifts measured compression from 3.310 to 3.751 for DRHE and 3.385 to 3.697 for LPC — still perfectly lossless. For DRHE the fix costs 40 extra block RAMs (4% of the chip) and *nothing else*: same logic, same clock speed, same latency.

I have implemented, verified and synthesised the corrected versions of both algorithms. I also found and fixed two implementation bugs that would have shipped, and corrected three factual errors — one in the literature, two in my own Thesis A report.

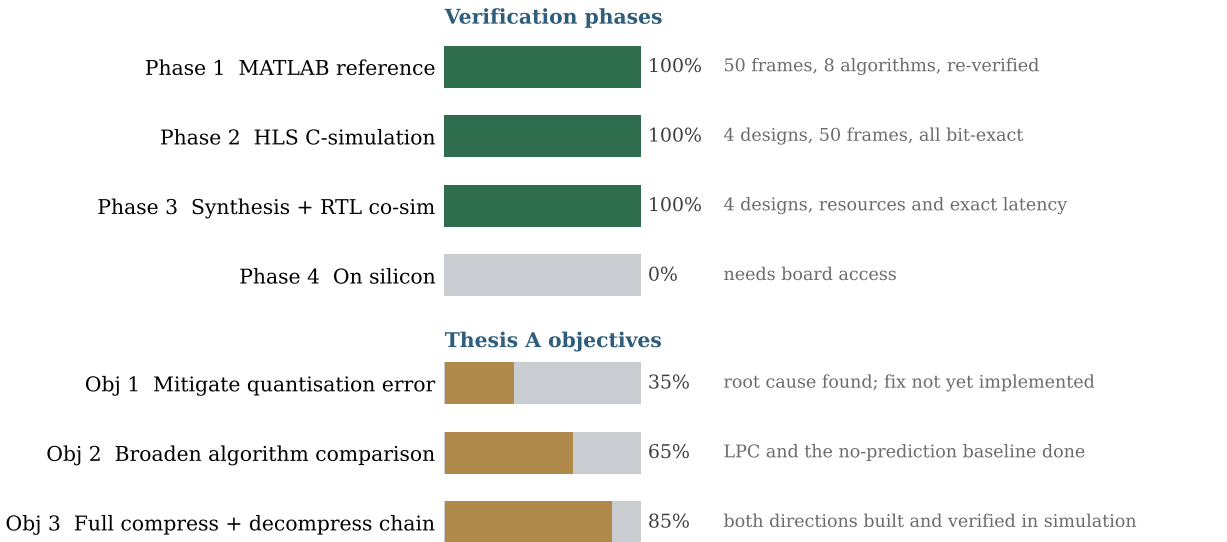


Figure 1: Where the project stands. Phases 1–3 are complete for all four designs; Phase 4 needs the physical board.

1 What I built

1.1 The two compressors, in plain terms

Both algorithms work the same way in outline. They *predict* what the next radar sample will be from samples already sent, then transmit only the *difference* between the prediction and reality. When the prediction is good the differences are tiny, and tiny numbers can be written in very few bits. Nothing is thrown away — the receiver runs the same prediction and adds the difference back — so the process is exactly lossless.

They differ in how they predict:

- **DRHE** converts each sample to magnitude and phase and predicts each with a fixed filter. It assumes a target's brightness stays roughly constant and its phase advances by a constant step, which is what a target moving at constant speed does. It is *streaming*: it needs no buffer and starts producing output immediately.
- **LPC** fits a mathematical model to the data instead of assuming one. It is more flexible, but it has to see the whole frame before it can fit the model — so it needs a 3.1 Mibit buffer and cannot output anything until the frame is complete.

Both then feed their differences into the same entropy coder, which assigns short bit patterns to common values and long ones to rare values.

1.2 Deliverables

- DRHE compressor and decompressor in Vitis HLS (C++), synthesised for xcku5p.
- LPC compressor and decompressor, likewise.
- TDM-corrected versions of both, kept alongside the originals so every comparison is like-for-like.
- A 50-frame testbench driven by real ColoRadar data exported from MATLAB.
- An 8-case synthetic edge-case testbench.
- Four MATLAB audit scripts that produced the findings in Section 4.
- A 70-page written report covering all of the above.

2 How I verified it

The single most important property of these compressors is that they are *exactly* lossless — a radar that silently loses a weak target to save bandwidth is not safe. So the verification had to be able to *prove* exactness, not estimate it. I used a four-stage ladder, each stage checking the one above it at a lower level:

1. **MATLAB, double precision.** The golden reference. Tells us exactly what bits should come out.
2. **HLS C-simulation.** The hardware C++ compiled and run as ordinary software. Asks: does it reproduce MATLAB *bit for bit*?
3. **RTL co-simulation.** The synthesised hardware, simulated cycle-by-cycle. Asks: did synthesis change anything? Also gives exact latency.
4. **On silicon.** Not yet done — needs the board.

Every design passes stages 1–3 with **zero** reconstruction error over all 50 frames.

Agreement with MATLAB is tighter than I set as the pass criterion. For LPC, a MATLAB model that accounts for AXI packet padding reproduces the hardware bit count on 50 of 50 frames to within 5×10^{-6} — meaning the underlying bit counts are identical and the only difference is packet padding.

3 Results before the audit

3.1 MATLAB benchmark (all 8 algorithms, 50 frames, 16 RX)

Table 1: Detection metrics are against a double-precision reference. *FN* is missed detections; *FDR* is what fraction of reported detections were spurious.

Algorithm	CR	FN%	FDR%	
FP16	1.00	0.00	0.00	reference-quality, no compression
FX16	1.00	3.77	1.11	the quantisation step alone
DRHE	3.30	3.77	1.11	lossless
LPC	3.37	3.77	1.11	lossless
BAQ	3.76	10.44	9.69	lossy — note the detection cost
RDVLE	1.12	24.01	4.62	lossy
RDVLE_UDRE	2.56	28.73	4.44	lossy

Two points worth making to anyone reading this table:

- **DRHE, LPC and FX16 have identical detection metrics by construction**, not by coincidence. All three reconstruct the quantised data perfectly, so the detector sees identical input. What those rows actually measure is the cost of the *quantisation* step, not of compression.
- **BAQ shows why lossless matters**. It reaches a slightly better ratio but misses nearly three times as many real targets and invents nearly nine times as many false ones. That is not a trade worth taking in a braking system.

3.2 Hardware (synthesis, xcku5p at 100 MHz)

	DRHE	LPC	Kiem's design
LUT	115,532	80,528	45,892
Flip-flops	57,105	17,393	9,243
Block RAM	40	256	10
DSP slices	132	168	44
Max clock	81.9 MHz	85.4 MHz	100 MHz
Frame latency	61,253 cycles	197,277 cycles	—
Frame buffer	none	3.1 Mibit	—

The surprise here was that **LPC is the cheaper design**, not the more expensive one. I had expected its division to dominate. It does not — the division runs 1,024 times per frame and accounts for 0.26 % of the cycles. What actually costs is DRHE's four floating-point transcendental functions (`sqrt`, `atan2`, `sin`, `cos`) evaluated for every sample of every channel. The real trade is **logic (DRHE) against memory and latency (LPC)**.

Neither design closes timing at 100 MHz. Both fixes are identified (Section 9).

4 The main finding: we were predicting along the wrong axis

4.1 How I found it

At this point every test passed and the compression ratios matched the literature, so the obvious conclusion was that the work was done. Before writing it up I ran one extra check that is not part of any standard compression evaluation:

What happens if I remove the predictor entirely and code the raw data through the same entropy coder?

Anything that survives is the entropy coder's doing, not the predictor's.

Configuration	bits/value	CR	vs. no prediction
No prediction at all	4.586	3.489	—
DRHE as implemented	4.821	3.319	$0.951\times$ (<i>worse</i>)
LPC as implemented	4.710	3.397	$0.974\times$ (<i>worse</i>)

Both predictors were *costing* bits. An independent check confirms it: the information content (order-0 entropy) of the raw data is 4.094 bits, and of the DRHE differences 4.538 bits. The predictor was *increasing* the entropy of data it was supposed to be simplifying.

4.2 Why

Our radar has 12 transmit antennas that fire one at a time in rotation. When the data is laid out for a streaming compressor, the “ramp” index is

$$m = (\text{chirp loop}) \times 12 + (\text{transmitter index}),$$

so the transmitter changes *every single ramp*. Ramp m and ramp $m - 1$ were transmitted by different antennas, in different physical positions. The difference between them is dominated by *where the target is in angle*, not by how fast it is moving — and it jumps around on a period-12 pattern that neither algorithm can track.

The genuine slow-time neighbour — the one whose difference reflects target *motion* — is ramp $m - 12$.

This is not a bug in Kiem’s work. On a single-transmitter radar, which is what his specification addresses, ramp $m - 1$ is the correct neighbour. The error is in applying that specification unmodified to a MIMO sensor of a different kind — which is what I did before measuring it.

4.3 What it looks like on real data

Figure 2 shows sixteen consecutive ramps of one range bin, run through the same DRHE predictor twice. Only the prediction lag differs.

The underlying reason is visible in the raw numbers. Against ramp $m - 1$, the target’s phase step swings over ± 2.6 radians with no pattern. Against ramp $m - 12$, it is 0.002, -0.040 , 0.017, -0.008 — essentially zero, because it is a stationary target and twelve ramps later the same antenna sees the same thing.

4.4 The fix, and what it costs

Table 2: Corrected designs against their originals. Vitis HLS C-simulation, all 50 frames. Everything remains exactly lossless.

Design	CR	MaxDiff	LUT	FF	BRAM	Latency (cyc)
DRHE, original	3.30987	0	115,532	57,105	40	61,253
DRHE, corrected	3.75082	0	115,736	57,263	80	61,253
LPC, original	3.38497	0	80,528	17,393	256	197,277
LPC, corrected	3.69681	0	86,298	23,154	192	325,166

- **DRHE: 13.3 % compression for 40 block RAMs.** Logic, DSPs, clock speed and measured frame latency are all unchanged. The extra memory holds one copy of the predictor’s state per transmitter. This is about as close to a free improvement as hardware gets.
- **LPC: 9.2 %, and a genuine trade.** Restructuring for the deeper lags happened to fix an unrelated bottleneck — block RAM drops 25 % and every loop now runs at full rate, which the original never achieved. But the frame takes $1.65\times$ longer. I have identified how to recover most of that.

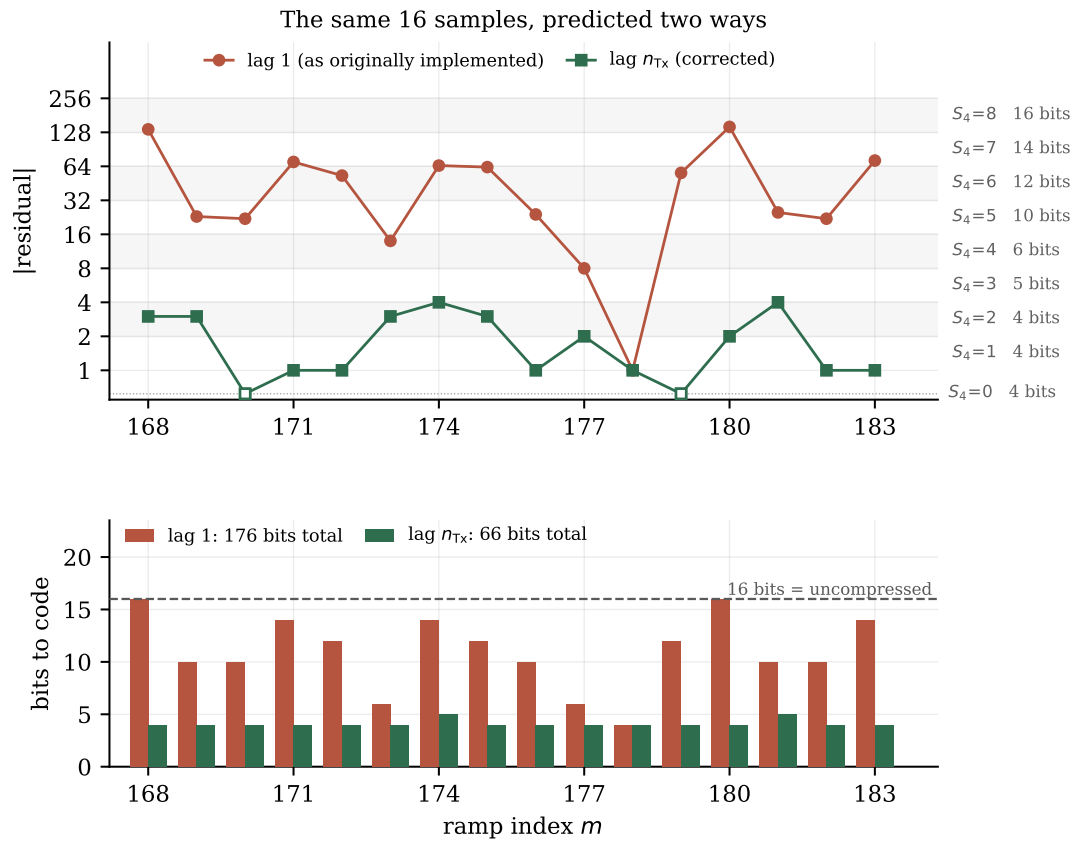


Figure 2: The same sixteen real samples, predicted two ways. Above: the size of each difference, with the bit-cost bands of the entropy coder marked. Below: what each sample costs. Predicting from the previous ramp spends 176 bits; predicting from the previous ramp of the same transmitter spends 66 — for identical information.

Figure 3 shows the improvement is uniform, not an average artefact — the corrected designs beat their originals on every single frame.

5 Second finding: what a compression ratio actually measures

The same audit answered a question I had not thought to ask: *where does the 3.5 come from, if not from prediction?*

- **A factor of 1.63 is unused container headroom.** Our quantisation step scales the data assuming a full-scale ADC input. The ColoRadar recording sits far below that — its peak raw sample is 914 out of a possible 32,767, and after processing the peak 16-bit code is 436. **Only 9.8 of the 16 bits ever carry information;** the other 6.2 are zero in every sample. Any entropy coder removes them for free.
- **A factor of 2.14 is genuine.** Radar range-FFT data is sparse — most bins are noise near zero — and that is real, transferable compression.
- **The rest is prediction,** which as implemented was negative and after the correction is about 1.08.

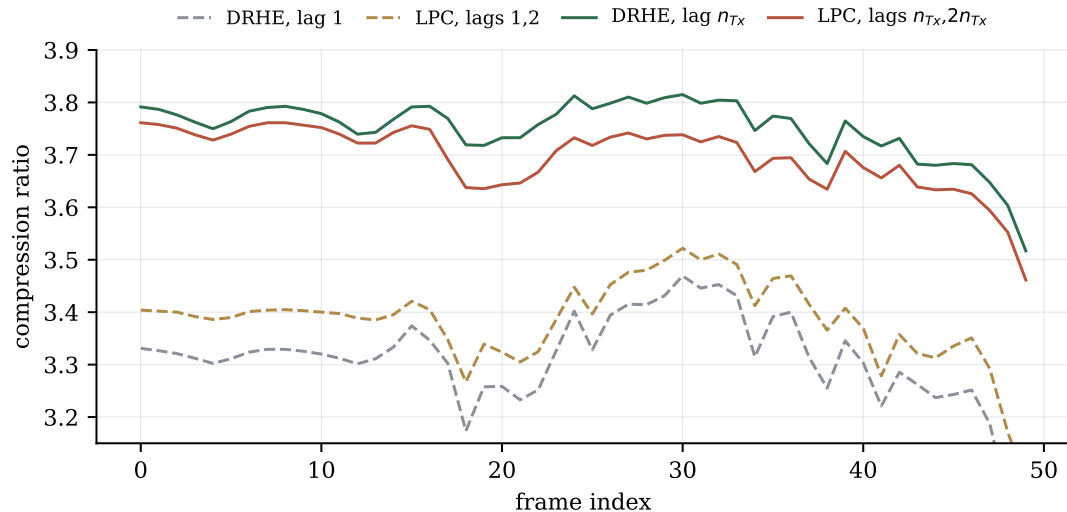


Figure 3: Per-frame compression ratio across the 50-frame sequence.

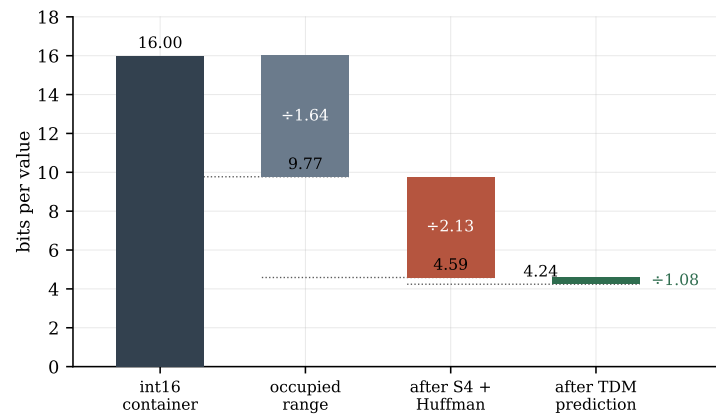


Figure 4: The compression ratio decomposed.

Why this matters beyond our project. It means the compression ratios reported in this field — ours and others' — are substantially a property of the *sensor's gain setting* rather than of the algorithm. A better-scaled recording would compress measurably less on identical code. I think reporting a no-prediction baseline alongside any such figure should be standard, and as far as I can find, it is not.

It also points directly at Thesis A Objective 1. We are currently wasting 6.2 bits of dynamic range at exactly the point where weak targets are being crushed into the quantisation noise.

6 Two bugs found and fixed

Both were invisible on real data and both were caught by the synthetic edge-case suite. Both would have shipped.

6.1 An unrepresentable value

The entropy coder's largest category holds exactly 2^{15} values, which exactly fills its payload field — leaving no room for the value -32768 , which 16-bit arithmetic can produce. Encoding it produces bit-for-bit the same output as $+32767$, and the decoder returns the wrong number. The edge-case test showed a reconstruction error of 65,535, which is precisely the distance between those two values.

The fix is to clamp -32768 to -32767 — but clamping alone is not enough. Once the

reconstruction differs from the original by even one bit, the encoder and decoder start predicting from different histories and *diverge*, with the error growing through every remaining ramp. The complete fix is to have the encoder reconstruct exactly what the decoder will see and predict from *that*. This costs pipeline depth but bounds the error to one sample.

6.2 A silent numerical failure

LPC’s solver needs products of large integers, computed as 128-bit values. Converting one of those directly to floating point is incorrect above 2^{64} in this toolchain — it returns zero. A determinant of 4.4×10^{20} was converting to -1.2×10^{18} , the wrong sign. Fixed by scaling all the operands down by a common shift before dividing, which leaves the answer unchanged.

On real data the numbers involved only reach 44 bits, so this was entirely unreachable with our dataset.

Methodological point worth recording. Fifty frames of real data — 157 million samples — found neither bug. An eight-case synthetic suite found both in minutes. Conversely, the synthetic suite would never have found the prediction-axis error, because synthetic data has no transmitter structure to get wrong. The two kinds of testing find different classes of problem and neither substitutes for the other.

7 Corrections to the record

Three factual errors surfaced during the audit. I mention them because two are mine and should be fixed before submission.

1. **A throughput figure in the literature is in the wrong units.** Kiem reports 11.92 Gbit/s. Read as decimal that is 119.2 bits per clock cycle on a 128-bit bus — an odd number. Read as *gibibits* it is 127.99, i.e. exactly 128 — the full bus width at one sample per cycle. The figure is binary; its decimal equivalent is 12.80 Gbit/s. Every comparison against 11.92 understates the gap by 7.4%.
2. **Our “false positive rate” is a false discovery rate.** It divides by the number of *reported* detections, not by the number of true negatives. It is a legitimate metric — arguably the more useful one here — but it is not what the name denotes.
3. **Two citations in the Thesis A executive summary are wrong.** The BAQ reference should be Kwok & Johnson, IEEE TGRS vol. 27 no. 4 pp. 375–383 (1989), on *synthetic* aperture radar; the name and page range currently cited do not correspond to any such paper. The ColoRadar reference should be IJRR vol. 41 no. 4 pp. 351–360 (2022), not vol. 40 (2021). Also, “simulated aperture radar” should read “synthetic” throughout.

Separately: the DRHE description in Thesis A §2.5.3 does not match what is actually implemented. It describes differencing *raw ADC* samples and coding with Golomb–Rice. The real algorithm operates on the *range-FFT output*, predicts in polar coordinates with two IIR filters, and codes with a fixed Huffman dictionary. Worth reconciling before submission.

8 Progress against the Thesis A objectives

Objective	Status	Notes
1. Mitigate quantisation error	35 %	Root cause now precisely characterised: only 9.8 of 16 bits are in use. An adaptive per-frame scale factor is a simpler and larger win than either approach proposed in Thesis A. Not yet implemented.
2. Broaden the algorithm comparison	65 %	LPC fully implemented and characterised in hardware; BAQ and RDVLE benchmarked in MATLAB; a no-prediction baseline added, which turned out to be the most informative comparison of all. NXP CP4/CP8 and AdaRadar not attempted.
3. End-to-end hardware	85 %	Compression <i>and</i> decompression implemented for both algorithms and verified bit-exactly through RTL co-simulation. Remaining: place-and-route, and running it on the physical board.

9 Next steps

In the order I propose tackling them.

9.1 High value, low effort

- 1. Adaptive FX16 scale factor.** Directly addresses Objective 1, recovers roughly 6 bits of dynamic range where weak targets currently die, and makes our reported compression ratios reflect the algorithm rather than the recording. Probably the single highest-value change remaining.
- 2. Fix DRHE's initiation interval.** The design currently accepts one sample every *two* cycles instead of every cycle, because of a memory port conflict caused by a reset that runs on one ramp in 192. Hoisting that reset out of the loop should roughly double throughput, to 11–12 Gbit/s — in line with Kiem.
- 3. Recover LPC's latency.** The corrected LPC makes two short passes where it used to make one long one, and pays a pipeline start-up cost 512 times over. Merging the passes should recover most of the 65,000 lost cycles.

9.2 Medium effort

- 4. Replace the polar conversion with a fixed-point CORDIC.** The four floating-point transcendentals dominate DRHE's logic usage *and* its critical path, so this addresses both the resource count and the 81.9 MHz clock ceiling in one change.
- 5. Re-derive the Huffman dictionary.** After the correction, 86 % of differences fall into the three cheapest categories, which is a more concentrated distribution than the current fixed dictionary assumes. A dictionary matched to the corrected data is a cheap few per cent.
- 6. Place-and-route, then Phase 4 on the board.** Real $F_{\max}$, real resource counts, power measurement, end-to-end latency.

9.3 If time allows

- Benchmark against NXP CP4/CP8 and AdaRadar, completing Objective 2.
- Check whether the prediction-axis issue has an analogue in Doppler-division or code-division MIMO, which distribute the slow-time structure differently.

10 Risks and open questions

- **Board access is the critical path for Phase 4.** Everything else can proceed in simulation, but power and real timing cannot.
- **Timing closure.** Neither design meets 100 MHz today. The CORDIC change should fix it; if it does not, the fallback is to report an 80 MHz clock honestly, which still leaves useful throughput.
- **All hardware numbers are pre-implementation estimates.** Post-route $F_{\max}$ is typically lower than the synthesis estimate.
- **Scope.** Objective 2 is only partly done. I would rather finish the quantisation work (Objective 1) properly than add two more algorithms to the comparison — but that is a judgement call worth discussing.

Things I would like your view on

1. **Is the prediction-axis finding worth writing up separately?** As far as I can establish it is not reported anywhere, it applies to any slow-time predictive coder on a TDM-MIMO radar, and the check that exposes it costs about twenty lines of MATLAB.
2. **Should I prioritise the adaptive quantisation work over widening the algorithm comparison?** My instinct is yes — it is the original Objective 1, it is now a well-characterised problem rather than a vague one, and it would improve the data every algorithm sees.
3. **What is the realistic timeline for board access?**
4. **How should the Thesis A corrections be handled** — a corrigendum, or simply fixed in the Thesis B document?

Where everything lives

<code>ThesisA/Matlab_Sim/</code>	MATLAB pipeline, benchmarks and audit scripts
<code>ThesisA/hls_component/</code>	all four HLS designs, testbenches, build configs
<code>ThesisA/paper/</code>	the 70-page written report and its figures
<code>ThesisA/drhe_testing_plan.md</code>	full test plan with every result recorded inline

Everything is committed to git, with the audit and the correction in their own commits so the before/after comparison is reproducible.